



Manual de Desarrollador: Radar de Precios Gunpla Chile

1. Visión General del Sistema

El proyecto es una aplicación monolítica construida en Python con Flask. Utiliza un enfoque tradicional de renderizado del lado del servidor (SSR) mediante Jinja2. La persistencia de datos se maneja a través de una base de datos local SQLite3 sin dependencias externas, y la recolección de datos se realiza mediante scripts de web scraping programados o ejecutados manualmente.

2. Arquitectura de Archivos y Carpetas

Basado en la estructura actual del proyecto, el código está organizado bajo un patrón modular (MVC adaptado para Flask):

Directorio / Archivo	Función Principal
app.py	El corazón de la aplicación. Inicia el servidor Flask, registra los <i>Blueprints</i> (rutas) e inicializa el planificador (APScheduler) si se configura.
config.py	Contiene variables globales de configuración (nombre de la base de datos, credenciales por defecto, configuraciones de Flask).
.env	Variables de entorno seguras (no se suben al repositorio) para contraseñas o claves secretas.
precios_comunidad.db	La base de datos SQLite donde reside toda la información.
core/	Lógica de negocio independiente de la web. Contiene database.py (conexión, ejecución de consultas SQL y lógica de fusión) y logger.py (registro de errores y eventos).
routes/	Controladores (Blueprints). views_inicio.py maneja la página principal con sus filtros y paginación. views_producto.py renderiza los gráficos de cada Gunpla. admin.py y admin_login.py controlan el panel privado.
scrapers/	Scripts aislados de extracción de datos. blaster.py y hangar.py contienen la lógica específica para leer el HTML de esas tiendas utilizando BeautifulSoup4.
static/	Archivos públicos como hojas de estilo CSS, scripts JS del lado del cliente y la carpeta de imágenes del carrusel.
templates/	Vistas en formato HTML/Jinja2. Incluye base.html (plantilla maestra), inicio.html, producto.html y las vistas de administración.
ejecutar_manual.py	Script de utilidad para forzar la ejecución de los scrapers desde la consola sin levantar el servidor web.
reset_catalogo.py	Script de mantenimiento para limpiar o reiniciar tablas específicas de la base de datos.

3. Modelo de Datos (Base de Datos)

El sistema utiliza una estructura relacional diseñada para normalizar productos de distintas tiendas.

catalogo_global: La tabla maestra. Guarda el identificador único, nombre_estandar y el grado (PG, MG, HG, etc.) del producto.

publicaciones_tiemdas: Actúa como puente. Relaciona un producto específico de una tienda (con su propio ID local) con el ID del catalogo_global.

historial_precios: Almacena cada captura de precio. Contiene el precio extraído, la fecha exacta de captura y la URL de origen. Vinculada a publicaciones_tiemdas.

carrusel: Guarda el nombre de los archivos (imágenes) que se muestran en el banner superior.

estadisticas y logs: Tablas de telemetría para contar visitas y registrar acciones del administrador.

4. Patrones de Diseño Utilizados

Función Maestra de Base de Datos

Para evitar abrir y cerrar conexiones manualmente en cada archivo, se diseñó la función *execute_query(query, args, ...)* en *core/database.py*.

Regla de Desarrollo: Toda consulta SQL (SELECT, INSERT, UPDATE, DELETE) en cualquier parte del sistema debe pasar obligatoriamente por esta función.

Manejo de Importaciones Circulares

Para funciones complejas que involucran múltiples herramientas del core (como fusionar productos y registrar la acción en el log simultáneamente), las importaciones se realizan de manera diferida (dentro de la función misma) para evitar que los módulos bloqueen el inicio de la aplicación.

Blueprints de Flask

En lugar de tener todas las rutas (*@app.route*) en *app.py*, la aplicación está fragmentada usando *Blueprints*.

Regla de Desarrollo: Si deseas crear una nueva sección web (por ejemplo, */estadisticas*), debes crear un nuevo archivo en la carpeta *routes/*, definir el Blueprint, y luego registrarlo en *app.py*.

5. El Frontend (Interfaz de Usuario)

Motor de Plantillas (Jinja2)

Se utiliza herencia de plantillas. `base.html` contiene el `<head>`, la barra de navegación y los estilos base. Todas las demás vistas heredan de esta usando la etiqueta `{% extends "base.html" %}` e inyectan su contenido en el bloque `{% block content %}`.

Sistema de Estilos y Modo Oscuro

No se utilizan frameworks CSS externos (como Bootstrap o Tailwind). El diseño es 100% *Vanilla CSS* basado en Variables Nativas (`var(--nombre-variable)`). El modo oscuro funciona mediante un script ligero en JavaScript que intercepta un clic en el botón y añade una clase `.dark-mode` a la etiqueta `<body>`, alterando el valor de todas las variables CSS instantáneamente.

Gráficos

El análisis visual de precios en `producto.html` está delegado a `Chart.js`. El servidor Flask envía una lista de fechas y una lista de precios a la vista, y estas son inyectadas en JavaScript como arreglos para renderizar el gráfico lineal de fluctuaciones.

6. Lógica de Paginación y Filtrado

Implementada en `routes/views_inicio.py`. La lógica sigue una secuencia estricta por razones de rendimiento:

Captura de parámetros: Obtiene la página actual, el límite (`per_page`), el término de búsqueda (`q`), el filtro de grado (`grado`) y el orden (`sort`) desde la URL (método GET).

Construcción dinámica: Construye la cláusula `WHERE` y `ORDER BY` en SQL basándose en los parámetros.

Conteo total (Count): Realiza una consulta rápida solo para contar el total de filas que coinciden con los filtros. Esto permite calcular matemáticamente cuántas páginas existirán.

Consulta principal con Offset: Realiza la consulta completa (con cruces de tablas para buscar el último precio activo) aplicando `LIMIT` y `OFFSET` para extraer únicamente (por ejemplo) los 24 productos de la página 3, reduciendo la carga de memoria a casi cero.

7. Mantenimiento y Extensión

Para agregar una tienda nueva:

Crea un nuevo script en la carpeta scrapers/ (ej: tiendax.py).

Analiza el HTML objetivo y programa la extracción del título, precio y URL.

Conecta el output de tu script a las funciones de inserción de catálogo en core/database.py.

Para limpiar datos duplicados:

Utiliza el panel de administración web (ruta /admin). Su herramienta de fusión permite tomar el ID de un producto repetido y traspasar todas sus recolecciones de precio históricas a un ID principal estandarizado, eliminando luego el registro sobrante.